Small Door

Senior Data Engineer Case Study

By: Renaud Beaupre

Made with GAMMA

What We Built

Single-source dataset → purpose-built **dbt stack on DuckDB**. All four dashboard questions answerable from `appointments` with **no joins at query time.**

Model	Type	Purpose
stg_clinic_appointments	staging view	Type casts, renaming, quality flags
`date_spine`	ref table	Date reference 2015–2030
appointments	marts table	OBT — primary BI surface
`patient_summary`	marts table	Patient-level aggregate, standalone
no_show_by_lead_time	analysis table	No-show × lead-time × SMS

Made with GAMMA

Data Quality

Principle: flag and retain, never silently drop or impute. Issues surfaced as boolean flag columns — analysts choose whether to filter. Dropping rows hides problems; imputing invents facts.

Issue	Volume	Decision
Age = −1	1 row	Keep raw, age_invalid=true
Age = 0	3,539 rows	Keep, no flag
Age > 110	5 rows	Keep, no flag
Negative lead time	38,568 (~35%)	Keep raw, lead_time_valid=false
Small clinics (n=1–2)	~5 clinics	Keep all rows
Disability 0–4	2,241 rows	Keep as integer ordinal

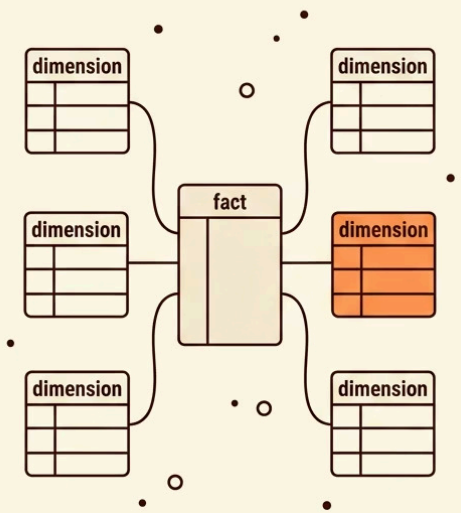
Made with GAMMA

Key Modelling Decisions

Decision	Chose	Rejected
No dim_patient_history	Attributes direct from source row	SCD2 with BETWEEN join
No `dim_clinic`	`clinic_name` direct from staging	1-column lookup table
date_spine joined on date	Direct equality join	Compute date attributes inline
`sms_sent` as boolean	Flag on OBT	`dim_sms`
Disability as integer 0–4	Raw ordinal value	Cast to boolean
Utilization proxy	`is_utilized = NOT no_show`	Capacity-based rate

Made with GAMMA

Hard Call 1 — OBT vs Star Schema



Star Schema

Multiple sources

Needs SCD2 for patient history

Fan-out joins at query time

More models & tests

OBT chosen

Single table clinic_appointments

Each row carries state at scheduling time

Zero joins required

4 models total

Why OBT?

Star schema solves an **integration problem** — joining entities across multiple source systems. Here there is no integration problem: one source table, one grain, all dashboard questions answerable with **no joins**.

What would change our mind? Newintegration of a separate EHR or patient registration system — then dimensional modelling becomes necessary.

Made with GAMMA

Hard Call 2 — Patient Attributes on Each Row vs SCD2

Decision

Carry patient attributes (gender, age, condition flags) **directly from staging** rather than building a separate `dim_patient_history` SCD2 table.

Why direct sourcing

Each appointment row already records patient state **at scheduling time** — that is the history. SCD2 would reconstruct point-in-time accuracy the source already provides natively.

> Would change our mind: confirmation that attributes are **backfilled to current values** at extraction time — then every historical row is silently wrong and SCD2 becomes mandatory.

Made with GAMMA

Hard Call 3 — Retain Negative Lead Times

38,568 rows (~35%) have `scheduled_at > appointment_at`. Evidence points to a **timezone artifact**: max magnitude −6.6 days, ScheduledDay always has time component, AppointmentDay always midnight UTC.

Drop rows

Silently biases no-show counts

Null lead_time_days

Loses distribution; can't inspect

Swap dates

Assumes intent we don't have

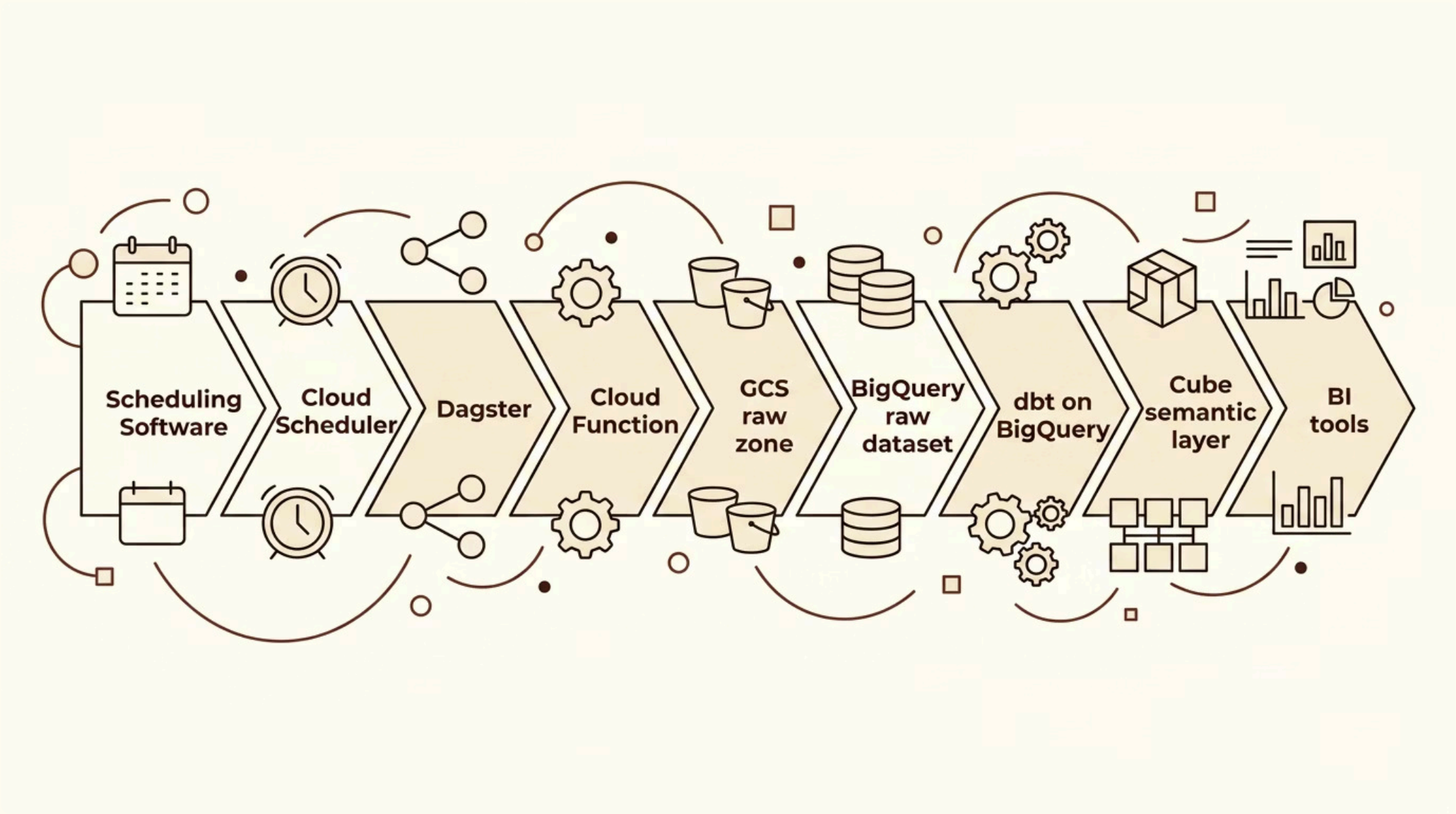
✓ Flag & retain

`lead_time_valid=false` — excluded from no-show analysis by default

Invalid rows have **~5% no-show rate** vs **~29%** for valid — two distinct populations of behaviour. Blending both without acknowledgement produces a misleading overall rate.

Made with GAMMA

Production Architecture



Q3 — What breaks first on source schema change? `stg_clinic_appointments` — the only model referencing source column names. A rename requires a fix in one place. Dagster fails the staging asset and **blocks all downstream models automatically.**

Made with GAMMA

Testing & Freshness

Q2 — How do you test the numbers?

- **Ingestion:** Cloud Function publishes row count to Cloud Monitoring; alert on drop vs rolling average
- **dbt:** unique, not_null, range checks as Dagster asset checks
- **Dagster:** Row count assertions between raw and mart layers

Q4 — How does an analyst know data is stale?

`dbt source freshness` in `sources.yml` using `updated_at` as `loaded_at_field` — serves double duty as incremental watermark and freshness signal.

```
warn_after:  {count: 12, period: hour}
error_after: {count: 25, period: hour}
```

If source is stale, **Dagster aborts** before any transforms run.

Made with GAMMA

Metric Governance

Operations and Finance will want different definitions of **"no-show rate."** Solution: **Cube** as the semantic layer in front of BigQuery.

Single access point

All BI tools query through Cube — no direct BigQuery access

Named measures

`no_show_rate` variants declared, version-controlled, peer-reviewed, one owner per definition

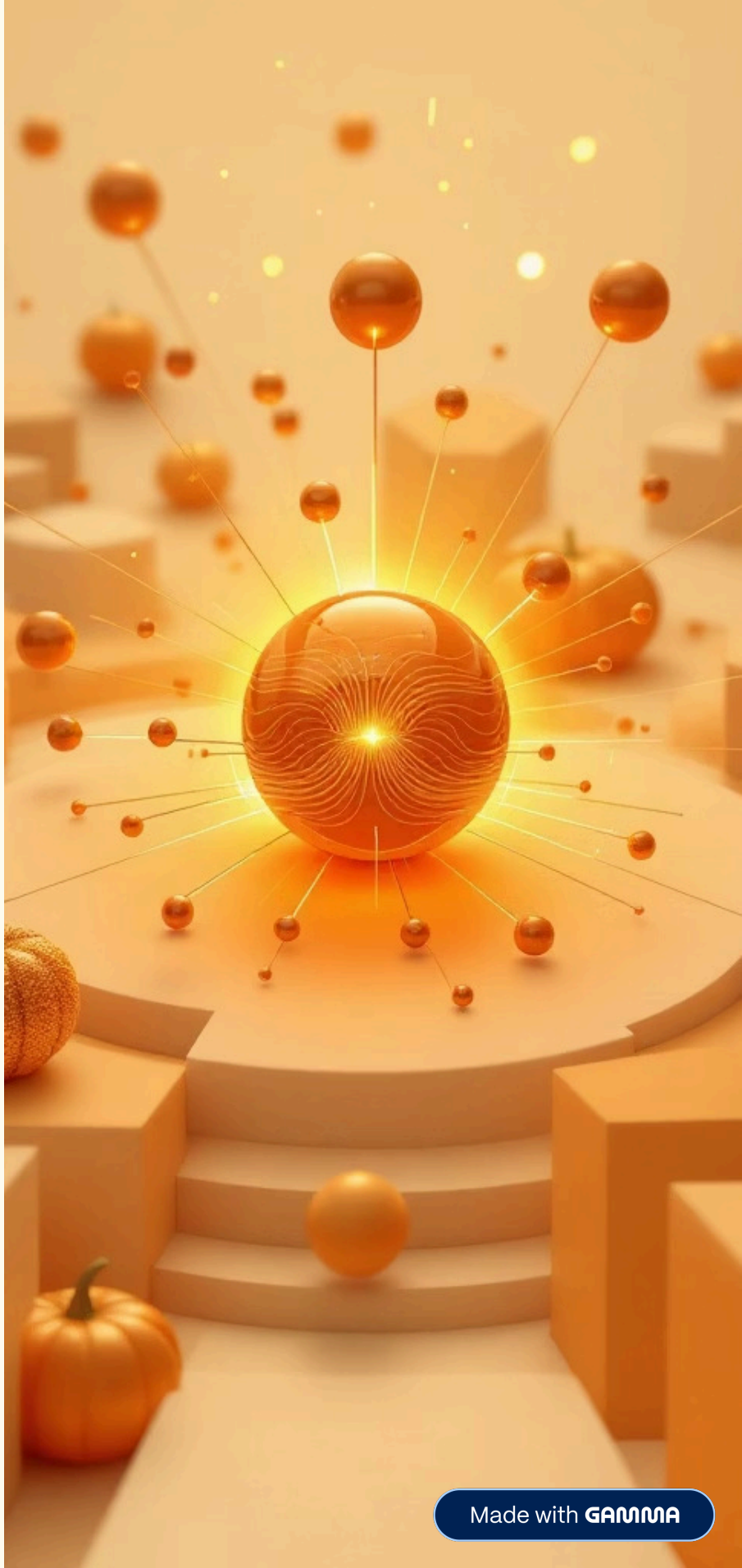
No shadow metrics

Analysts cannot invent a third definition in a calculated field

Domain separation

If definitions diverge at the data level, a separate dbt mart is built for that domain

The principle throughout: every metric definition has a **single owner, a written spec, and a version-controlled implementation** — never a spreadsheet or a BI tool calculated field.



Made with GAMMA